

Mizan.ai — Documentation ↔ Homepage Chatbot Integration

Technical reference. How frontend/documentation.html is wired into the homepage assistant's system prompt, and how documentation.html itself was structured/built. English-only — this is an internal engineering note, not a customer-facing document. As of 2026-08-06.

1. How the Documentation page is linked to the chatbot

No RAG / vector search is used. The full English half of frontend/documentation.html (~14.8k characters / ~2,500 words / ~3,700 tokens) is parsed to plain text once at process import time and spliced directly into the system prompt sent on every chat turn. At this size, giving the model the whole page outperforms retrieval: no chunking/embedding pipeline, no relevance-threshold tuning, no risk of the wrong chunk winning — the model just has everything, every time.

1.1 Request path

- Browser → **POST /api/chat** (app/main.py:293) → chatbot_agent.generate_reply(message, language=None, session_id, user_id).
- **ChatbotAgent** (features/chatbot/langgraph_chatbot.py) is a module-level singleton — instantiated once at app/main.py import time (line 52: chatbot_agent = ChatbotAgent()), not per-request.
- generate_reply() wraps the message in a LangGraph chatState and invokes the compiled graph with config={"configurable": {"thread_id": session_id, "user_id": user_id}}.

1.2 LangGraph pipeline (_build_graph)

route_intent → sync_memory → generate_reply → trim_memory → END

Checkpointed via PostgresSaver (falls back to in-memory MemorySaver if Postgres is unavailable). Documentation content enters at the **generate_reply** node, inside _build_wire_messages(), which builds the system message via build_system_prompt(language, profile).

1.3 Extraction — features/chatbot/langgraph_chatbot.py

Module-level constants and the extractor class, quoted verbatim:

```
_DOCUMENTATION_PATH = Path(__file__).resolve().parents[2] / "frontend" / "documentation.html"
_DOC_BLOCK_TAGS = {"p", "li", "tr", "h2", "h3", "h4"}
```

```
class _DocTextExtractor(HTMLParser):
    def __init__(self):
        super().__init__(convert_charrefs=True)
        self.parts: List[str] = []
        self._skip_depth = 0

    def handle_starttag(self, tag, attrs):
        if self._skip_depth or dict(attrs).get("class") == "mz-lang-switch":
            self._skip_depth += 1
            return
        if tag in _DOC_BLOCK_TAGS:
            self.parts.append("\n")
        if tag in ("th", "td"):
            self.parts.append(" | ")

    def handle_endtag(self, tag):
        if self._skip_depth:
            self._skip_depth -= 1

    def handle_data(self, data):
        if self._skip_depth:
            return
        text = data.strip()
        if text:
            self.parts.append(text + " ")
```

```
def _load_documentation_text() -> str:
    try:
        raw = _DOCUMENTATION_PATH.read_text(encoding="utf-8")
        start = raw.index(">", raw.index('id="content-en')) + 1
        end = raw.rindex("<div", 0, raw.index('id="content-ar'))
    except (OSError, ValueError):
        return ""
    extractor = _DocTextExtractor()
    extractor.feed(raw[start:end])
    text = "".join(extractor.parts)
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n\s*\n+", "\n", text)
    return text.strip()

FEATURE_DOCUMENTATION = _load_documentation_text() # run once, at import time
```

Mechanics:

- **Path resolution** is relative to this module's own file location (`Path(__file__).resolve().parents[2]` = repo root, since `langgraph_chatbot.py` sits at `features/chatbot/`), not CWD — robust regardless of how/where `uvicorn` is launched from.
- **Slice boundary**: everything between the opening tag's `>` that follows `id="content-en"`, and the `<div` immediately preceding `id="content-ar"` — i.e. the English content block only (see §2.3 for why the file has these two IDs).
- **Block-tag set** `{p, li, tr, h2, h3, h4}` triggers a newline on open; `th/td` get a `" | "` separator so table rows stay readable as one line each once flattened.
- **mz-lang-switch skip**: the extractor tracks a depth counter (`_skip_depth`) so the small in-page "switch language" link — UI chrome, not documentation content — and anything nested inside it never reaches `handle_data`.
- **Whitespace cleanup**: collapses runs of spaces/tabs, then collapses multiple consecutive blank lines to one.
- **Failure mode**: `OSError` (file missing — e.g. image not yet rebuilt after a `documentation.html` edit) or `ValueError` (marker not found — unexpected markup) both caught → returns `""`. The prompt-builder below treats that as "no doc block", not a crash.

1.4 Prompt assembly — `build_system_prompt()`

```
doc_block = ""
if FEATURE_DOCUMENTATION:
    doc_block = f"""
```

```
Feature documentation (Mizan.ai's own
Documentation page, numbered sections —
use this to answer what a feature does
or how to use it):
{FEATURE_DOCUMENTATION}
```

```
When you answer using the documentation above,
mention the specific section number and title
so the user can read more themselves (e.g. "see
section 4.2 Prepare a period VAT return in the
Documentation page"; template given for Arabic
replies too, phrased as "see section <N> in the
Documentation page" in Arabic).
Only cite a section when you actually used it
to answer — not for small talk or unrelated
replies."""
```

This block is interpolated into the full system prompt string (also containing the rule list, the "free trial memory" disclosure line, the language directive, and the user-profile line if one exists), returned by `build_system_prompt(language, profile)`.

1.5 What actually goes to the model — `_build_wire_messages()`

```
def _build_wire_messages(self, state, language, profile):
    wire = [{"role": "system", "content": build_system_prompt(language=language, profile=profile)}]
    for message in state.get("messages", [])[-8:]:
        if isinstance(message, HumanMessage):
            wire.append({"role": "user", "content": str(message.content)})
        elif isinstance(message, AIMessage):
```

```
wire.append({"role": "assistant", "content": str(message.content)})
return wire
```

Sent as `{"messages": wire, "max_tokens": 512, "temperature": 0.2}` via `requests.post(self.model_url, ...)` to the QwenLite endpoint (env `QWEN_LITE_URL` / `MIZAN_QWEN_LITE_URL` / `MODAL_QWEN_LITE_URL`, timeout from `MIZAN_CHATBOT_TIMEOUT_SECONDS`, default 600s). The system prompt (documentation included) is rebuilt and resent on every turn — LangGraph's checkpoint only persists the message list, not a cached prompt.

1.6 Deployment dependency

`services/chatbot/Dockerfile: COPY ../../ ./` copies the entire repo (build context is repo root per `docker-compose.yml`) into the image at `/app` — confirmed directly: `docker exec mizan-chatbot sh -c "ls /app/frontend/documentation.html"` resolves. Because `FEATURE_DOCUMENTATION` is read once at import time, the sequence for any future `documentation.html` change is:

```
docker compose build chatbot
docker compose up -d chatbot
```

Without the rebuild step, the running container keeps serving whatever `documentation.html` content was baked into its image at last build — a stale copy, not a live read of the host filesystem (no volume mount for `frontend/` in `docker-compose.yml`).

1.7 Live verification performed (2026-08-06)

- `docker compose build chatbot && docker compose up -d chatbot`, confirmed `documentation.html` present in the container.
- `python -c "from features.chatbot.langgraph_chatbot import FEATURE_DOCUMENTATION; ..."` — confirmed 14,862 chars extracted, section "4.2 Prepare a period VAT return" present verbatim, no leaked "switch language" text.
- `POST /api/chat` with "How do I prepare a period VAT return in Mizan.ai?" — reply cited "section 4.2 Prepare a period VAT return in the Documentation page" and gave steps matching that section.
- `POST /api/chat` with an Arabic question about document reconciliation — reply answered in Arabic and cited section 5 (Document Reconciliation) by name, correctly pulling from the English-only source text.
- `pytest tests/features/chatbot/test_langgraph_chatbot.py` — 5/5 passed (pre-existing suite, unaffected by this change).

2. How documentation.html's own content was created

2.1 Source material

Every real frontend page was read directly before writing any documentation text, to pull the actual current feature set, field names, and behavior rather than writing from memory:

```
app.html           - feature grid / entry points
know-vat-zatca.html - grounded Q&A chat
translator.html    - paragraph translation
calculator.html    - VAT Compliance Center landing
invoice-check.html - Job 1 (invoice compliance check)
period-return.html - Job 2 (period VAT return prep)
comparator.html    - document reconciliation
converter.html     - file conversion + PDF editor toolkit
```

2.2 Per-feature template (applied to every numbered section)

- **Purpose & why it's worth using** — concrete VAT/ZATCA/business stakes.
- **How it works** — plain language; "an AI engine" named only where genuinely used, no implementation detail.
- **What you'll be asked for** — an `mz-table` (Field | What to enter), matching each page's real form fields.
- **How to use** — numbered `<ol>` steps.
- **Example** — one realistic scenario in a `.mz-doc-example` callout.

2.3 Document structure / IDs

Two top-level content blocks, gated by a one-time language picker:

```
<div class="mz-doc-card mz-lang-gate" id="lang-gate">
  <button onclick="mzDocShowLang('en')">Continue in English</button>
  <button onclick="mzDocShowLang('ar')" dir="rtl" lang="ar">
    [Arabic label: "Continue in Arabic"]
  </button>
</div>

<!-- §1.3's extraction reads content-en only -->
<div id="content-en" style="display:none;">
  ... 7 numbered .mz-doc-card sections ...
</div>

<div id="content-ar" dir="rtl" lang="ar" style="display:none;">
  ... mirrored Arabic content, same numbering ...
</div>
```

Toggle logic (persisted via localStorage, key mz_doc_lang):

```
function mzDocShowLang(lang) {
  document.getElementById("lang-gate").style.display = "none";
  document.getElementById("content-en").style.display = lang === "en" ? "block" : "none";
  document.getElementById("content-ar").style.display = lang === "ar" ? "block" : "none";
  localStorage.setItem(MZ_DOC_LANG_KEY, lang);
  window.scrollTo(0, 0);
}
const savedLang = localStorage.getItem(MZ_DOC_LANG_KEY);
if (savedLang === "en" || savedLang === "ar") mzDocShowLang(savedLang);
```

Section numbering is literal text inside each heading (e.g. <h4>4.2 Prepare a period VAT return</h4>), not a CSS counter — this is exactly what makes the chatbot's citations possible: the numbers survive HTML-stripping as plain text (§1.3).

2.4 CSS additions — frontend/style.css

Class	Purpose
.mz-doc-card	Top-level bordered card, one per numbered feature (reuses .mz-card's look: accent top border, radius, shadow; max-width 980px).
.mz-doc-subcard	Nested shaded box (surface-2 background) for a sub-feature inside a card (e.g. 4.1/4.2/4.3, 6.1/6.2) — visually distinct from its parent.
.mz-doc-label	Small uppercase eyebrow text ("Purpose", "How it works", ...) marking each subsection within a card.
.mz-doc-example	Highlighted callout box (accent-soft background, accent side border) for the Example paragraph.
.mz-lang-gate	The initial language-choice screen (+ .mz-lang-gate-buttons for its button row).
.mz-lang-switch	The small "switch language" link — deliberately excluded from the chatbot's extracted text (§1.3).
h4 { ... }	Previously undefined in style.css (only h1/h2/h3 existed) — added for .mz-doc-subcard titles.
ol, ul { ... }	Explicit padding/spacing — numbered/bulleted lists previously had no rule at all in this stylesheet.

2.5 Entry point

frontend/app.html topbar nav: Documentation, inserted between the existing About and Contact us links.

2.6 Content size

Metric	Value
documentation.html total size	~34 KB (both languages combined)
English content (id="content-en") extracted	14,862 characters / ~2,500 words
Approx. tokens added to every chat system prompt	~3,700
Top-level numbered sections	7 (.mz-doc-card each)
Sub-sections (.mz-doc-subcard)	5 (3 under VAT Compliance Center, 2 under Convert & Edit Files)
Field-reference tables (mz-table)	9

2.7 Files touched (both parts of this change)

File	Change
frontend/documentation.html	New file. The Documentation page (7 sections, EN+AR, language gate).
frontend/style.css	New rules: .mz-doc-card, .mz-doc-subcard, .mz-doc-label, .mz-doc-example, .mz-lang-gate/-buttons/-switch, h4, ol/ul spacing.
frontend/app.html	Added the "Documentation" nav link.
features/chatbot/langgraph_chatbot.py	Added: html.parser/Path imports, _DOCUMENTATION_PATH, _DOC_BLOCK_TAGS, _DocTextExtractor, _load_documentation_text(), FEATURE_DOCUMENTATION, doc_block wiring in build_system_prompt().